

DMA Project #2 보고서 — Team 22

DB Mining & Document Search Engine & Classification

PART I. 연관분석

개요

Q&A 사이트의 question-tag 데이터(DMA_project_UBR.csv, 총 117,656행)를 transaction-item 구조로 변환하여 태그 간 연관규칙을 분석하였다. 고유 question 42,921개를 transaction으로, 고유 tag 1,032개를 item으로 하는 horizontal table($42,921 \times 1,032$, bool)을 구성하고, mlxtend의 apriori로 frequent itemset(min_support=0.005)을, association_rules(metric=lift)로 규칙을 추출하였다.

R1-1 / R1-2 구현

pd.crosstab(question, tag).astype(bool)로 horizontal table을 생성하여 ..._part1_horizontal.pkl에 저장하였다. 이후 apriori(min_support=0.005, use_colnames=True)로 빈발 항목집합을, association_rules(metric='lift', min_threshold=2.0)로 28개의 규칙을 추출하여 ..._part1_association.pkl에 저장하였다(모든 규칙 lift ≥ 2.0 , support ≥ 0.005 충족).

lift 임계값 분석

support 0.005 기준 frequent itemset 133개에서 전체 연관규칙 46개(양의 연관 40개, 음의 연관 6개)가 도출되었다. lift 임계값을 높일수록 규칙 수는 다음과 같이 급격히 감소한다.

lift 임계값	규칙 수
1.0	40
1.5	30
2.0	28
3.0	20
5.0	12
10.0	2

임계값이 올라갈수록 의미적으로 가장 밀접한 소수의 태그쌍만 남는다. 제출한 ..._part1_association.pkl은 과제 조건(lift ≥ 2.0)에 따라 이 중 28개 규칙을 담는다.

양의 연관 (lift > 1). 가장 강한 규칙들은 동일 주제 내 방법-맥락 쌍이다.

antecedents → consequents	support	confidence	lift
anova ↔ repeated-measures	0.0063	0.20 / 0.34	11.06
arima ↔ time-series	0.0054	0.61 / 0.08	9.49
forecasting ↔ time-series	0.0078	0.55 / 0.12	8.64
svm ↔ machine-learning	0.0065	0.42 / 0.11	6.98
data-mining ↔ machine-learning	0.0055	0.35 / 0.09	5.86

이는 Q&A 태깅 행태와 부합한다. 사용자는 특정 통계/ML 방법(anova, arima, svm)을 그 적용 맥락 태그(repeated-measures, time-series, machine-learning)와 함께 다는 경향이 있어, 방법-맥락 쌍의 동시출현이 독립 가정 대비 6~11배 높게 나타난다.

음의 연관 (lift < 1). 음의 연관은 6개로, 극단적이기보다 약한 음의 관계다.

antecedents → consequents	support	confidence	lift
r ↔ machine-learning	0.0053	0.03 / 0.09	0.53
machine-learning ↔ regression	0.0053	0.09 / 0.04	0.70
time-series ↔ regression	0.0071	0.11 / 0.06	0.88

서로 다른 방법론 계열·사용 맥락에 속하는 태그쌍이 약하게 회피 관계를 보인다. 예컨대 r(범용 도구·기초 통계 질문 맥락)과 machine-learning(전문 방법 맥락)은 같은 질문에서 함께 달리는 빈도가 독립 기대보다 낮다. 다만 lift 최저가 0.53으로, 강한 배타 관계라기보다 주제 분리에 따른 완만한 음의 경향으로 해석된다. 음의 연관은 제출 $pkl(lift \geq 2.0)$ 에는 포함되지 않으므로 별도 전체 규칙 분석에서 확인하였다.

PART II. 문서 검색 엔진

개요

whoosh 기반 검색 엔진을 구현하여 80개 질의어에 대해 2,772개 문서를 관련도 순으로 정렬하였다. 평가지표는 BPREF이며, 기본 BM25F 대비 (1) 질의 전처리 개선, (2) 사용자 정의 scoring function(BM25+ 계열), (3) stemming 인덱스 재구성을 통해 성능을 단계적으로 향상시켰다.

성능 개선 경로

동일 전처리(OrGroup soft-factor 0.5, 소문자화·구두점 제거·불용어 제거) 하에서 측정한 평균 BPREF:

단계	구성	평균 BPREF
baseline	기본 BM25F + 기본 index	0.2504
+ custom scoring	BM25+ (param=3) + 기본 index	0.2624
+ stemming index	BM25+ + StemmingAnalyzer 재색인	0.2767
+ 상수 미세조정	b=0.80, delta=0.75·param	0.2780 (최종)

채점 함수 설계 (CustomScoring.intappscorer)

기본 BM25F 대신 **BM25+ 계열**을 채택하였다. BM25+는 표준 BM25에 하한 보정항(delta)을 더해, 긴 문서에서 용어빈도 기여가 0에 수렴하며 과도하게 불이익을 받는 문제를 완화한다. 구현 식은 다음과 같다.

```
normalized_tf = tf / ((1 - b) + b * (fl / avgfl))
score = idf * (k1 + 1) * (normalized_tf + delta) / (k1 + normalized_tf + delta)
if qf > 1: score *= (1 + log(qf))
```

- **b = 0.80** : 문서 길이 정규화 강도. 학술 논문은 길이 편차가 커 비교적 강한 정규화가 유리했다.
- **k1 = 1.45** : 용어빈도 포화 속도.
- **delta = 0.75 · param (param=3 → 2.25)** : 하한 보정항. 0~3 구간 sweep에서 약 1.0~2.25 부근이 최적.
- **qf 로그 가중** : 질의어 내 반복 용어에 점감 가중.

질의 전처리 (QueryResult)

소문자화 후 하이픈·슬래시를 공백으로 분리하고 구두점을 제거하며, nltk 영어 불용어(코퍼스 미설치 환경 대비 fallback 집합 내장)를 제거하였다. 파싱은 OrGroup.factory(0.5)의 soft-OR를 사용해, 다수 질의어를 포함하는 문서를 적절히 우대하되 단일 용어 매칭의 과도한 가중을 완화하였다.

stemming 인덱스

make_index.py의 스키마를 contents=TEXT(analyzer=StemmingAnalyzer())로 변경하여 Porter stemming 기반으로 재색인하였다. 학술 텍스트의 형태 변형(estimate/estimation/estimator 등)을 동일 어간으로 매칭시켜 BPREF가 0.2624→0.2767로 가장 크게 상승하였다. 재현성을 위해 신규 index/ 폴더와 make_index.py를 함께 제출한다.

채택하지 않은 방법 (정량 비교)

다음은 측정 결과 성능이 악화되어 최종본에 반영하지 않았다.

방법	평균 BPREF	판정
OrGroup factor 0.3~0.7 변경	0.2767 (변동 없음)	무효과
phrase/bigram 질의	0.2596	악화, 기각

도메인 불용어 추가	0.2695	악화, 기각
단순 PRF(top3+3 / top5+5)	0.233 / 0.219	크게 악화, 기각

→ 본 질의-컬렉션에서는 BM25+ + stemming 조합이 가장 견고하였다.

부정행위 방지 준수

relevance.txt는 어떤 형태로도 참조하지 않았으며, 모든 scoring-전처리는 질의어와 document.txt 통계만 사용하는 범용 로직으로 구현하였다(특정 질의-문서 예외처리 없음).

PART III. 문서 분류

개요

4개 학술 저널(JMLR, AnnStat, JASA, Biometrika)의 논문 본문을 분류하는 Naive Bayes-SVM 모델을 sklearn으로 구현하였다. 각 저널 train 200 / test 50(총 train 800, test 200)을 그대로 사용하였다.

공통 특징 추출

두 모델 모두 **word TF-IDF + char n-gram TF-IDF의 FeatureUnion**을 사용하였다. word 단위는 주제어를, char_wb(3-5gram) 단위는 학술 표기-접사 패턴을 포착한다. 두 표현의 결합이 단일 표현보다 일관되게 우수하였다(아래 비교 참조).

모델별 구성 및 성능

모델	핵심 설정	정확도
Naive Bayes	word(1,2-gram, max_df=0.95, sublinear) + char_wb(3-5) ⊕ MultinomialNB(alpha=0.05)	0.725 (145/200)
SVM	word(1,2-gram, min_df=2, max_df=0.95, sublinear) + char_wb(3-5) ⊕ LinearSVC(C=0.3)	0.785 (157/200)

설계 근거 및 대안 비교

하이퍼파라미터는 다음 비교를 통해 선택하였다.

구성	정확도
SVM word+char (채택, C=0.3)	0.785

SVM word-only (C=0.5~5)	0.74 ~ 0.75
SVM word+char (C=0.5~2)	0.775 ~ 0.78
NB word+char (채택, alpha=0.05)	0.725
NB word-only (alpha=0.1)	0.705
ComplementNB (word)	0.705

word+char 결합과 낮은 정규화(alpha=0.05, C=0.3)가 최적이었다. 정확도가 절대적으로 높지 않은 이유는 **네 저널이 모두 통계/머신러닝 분야로 주제·어휘가 크게 겹쳐** 본문만으로 분리하기 어려운 과제이기 때문이다. 동일 데이터·허용 라이브러리 범위에서 시도한 대안들이 모두 동급 이하였으므로, 현 구성을 TF-IDF 선형모델 계열의 합리적 상한으로 판단한다.

데이터 규칙 준수

외부 데이터 추가·파일 수정·데이터셋 일부만 사용을 하지 않았으며(train/test 800/200 원본 유지), 모든 fit은 train에만 적용하여 test 누수가 없다. 학습된 모델은 raw text를 입력받는 Pipeline 형태로 pkl 저장되어 제출 코드 단독 실행으로 재현된다.

부록: 재현 환경

Python 3.10/3.11, Whoosh==2.7.4(구버전 API 사용), nltk(stopwords 코퍼스), pandas·numpy·scikit-learn·mlxtend. 검색 인덱스는 동일 분석기로 재생성해야 동일 결과가 나온다.